

LC 329: Longest increasing path in a Matrix

© Authored by: Arpit Raj, LNMIIT Jaipur | July 29, 2026

Deshaw / Samsung / Goldman

Problem Understanding

→ **start** → any cell

→ **move** → new cell if value greater

Eg:

[9 9 4]

[6 6 8]

[2 1 1]

we can go 2 → 6

but not 6 → 2

ans = 4

hence **Directed acyclic graph (DAG)**

Approach 1: Bruteforce

→ try all 4 increasing neighbours

1 + max(ans from neighbour)

problem → some cell recomputed

Eg:

1 → [4] → [7] →

$3 \rightarrow [4] \rightarrow [7] \rightarrow \dots$

(some cells are overlapping/recalculated)

$$O((m \times n) \times 4^{(m \times n)})$$

Approach 2: DFS + Memoization

hence what we can do is **DFS + memoization** which basically remembers the answer returned by a cell, so each cell visits at max 4 neighbours only

$$O(n \times m)$$

Implementation (C++)

```

class Solution {
public:
    vector<vector<int>> dp;
    int n;
    int m;
    vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

    int dfs(int r,int c,vector<vector<int>>& matrix){
        if(r<0 || c<0 || r>=n || c>=m) return 0;
        if(dp[r][c]!=-1) return dp[r][c]; //already computed

        int maxu=1; //khud ke cell ka contri

        for(auto &it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nc>=0 && nr<n && nc<m){
                if(matrix[nr][nc]>matrix[r][c]){
                    maxu=max(maxu,1+dfs(nr,nc,matrix));
                    //har direction me jaake dekhenge max kaha se mil rha hai
                }
            }
        }

        dp[r][c]=maxu;
        return dp[r][c];
    }

    int longestIncreasingPath(vector<vector<int>>& matrix) {
        n=matrix.size();
        m=matrix[0].size();
        int ans=1; //atleast 1 ka to length hoga
        dp.assign(n,vector<int>(m,-1));

        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                ans=max(ans,dfs(i,j,matrix)); //har ek start position se dfs krne l
            }
        }

        return ans;
    }
};

```